Quasar redshifts from Gaia DR3 XP coefficients

Project report · 05 November 2026 · prepared with Claude Code

Summary

Question. Do the coefficients of Gaia DR3 BP/RP (XP) spectra carry information on quasar redshift? SDSS DR16Q spectroscopic redshifts were matched to Gaia DR3 sources with published XP coefficients, and redshift was predicted from the coefficients with cross-validated models.

- **Yes, strongly.** On 36,402 quasars, a kNN on the first 20 BP + 20 RP coefficients reaches σ_{NMAD} = 0.0040 in $\Delta z/(1+z)$ with 88% of objects within 0.01(1+z); an MLP reaches 0.50% catastrophic outliers ($|\Delta z|/(1+z) > 0.2$). Gaia colours alone give $\sigma_{NMAD} \approx 0.18$ and ≈ 29% outliers.
- The signal is visible directly: stacked spectra show Lyα, CIV, CIII], MgII, Hβ+[OIII] and Hα moving along $\lambda_{rest}(1+z)$.
- Gaia's own QSOC redshifts (template χ^2 fit) remain the most precise (σ_{NMAD} = 0.0030) but have more catastrophic outliers (1.4%) than the best MLP (0.50%).
- **Not novel.** Refereed work already estimates quasar redshifts from XP spectra with SDSS-trained models, including directly from the Hermite coefficients (Sect. 6). The study was therefore not developed into a paper.

1 Approach

Quasar broad emission lines cross the 330–1050 nm XP window as redshift changes. Even at XP's very low resolution, their positions and the continuum shape should be encoded in the 55+55 Hermite coefficients. The test has two parts: (i) see the signal in the data, and (ii) measure how much redshift information the coefficients carry beyond broadband Gaia colours, against Gaia's QSOC as an external benchmark and a shuffled-redshift control.

2 Data

2.1 Cross-match (01_crossmatch.py)

SDSS DR16Q (750,414 quasars) lists a Gaia DR2 counterpart for 469,774 of them. DR2 ids of the 301,346 counterparts with $G_{DR2} < 20.5$ were uploaded to the Gaia archive (ESA TAP), mapped to DR3 with `gaiadr3.dr2_neighbourhood`, joined to `gaia_source`, and the DR3 neighbour closest to the SDSS position was kept (< 1″; median separation 0.048″). QSOC redshifts come from `gaiadr3.qso_candidates`.

A direct 1″ positional join was tried first, but the archive aborts anonymous jobs after 60 s, which made it impractically slow. It did complete for the first 190,000 DR16Q rows, which serves as a completeness check: the id-based route recovers 6,392 of the 6,395 XP sources found positionally (99.95%), always the same DR3 source.

DR16Q quasars	750,414
with Gaia DR3 counterpart < 1″ (G_DR2 < 20.5)	301,344 (40.2%)
… with XP coefficients	36,404 (4.85%)
… with Gaia QSOC redshift	36,356
usable after cleaning	36,402
G (5 / 50 / 95 percentile)	17.32 / 18.56 / 18.97
z (5 / 50 / 95 percentile)	0.43 / 1.50 / 2.67

Unlike the generic G < 17.65 limit for published XP spectra, 92% of the matched quasars with XP are fainter than that, down to G ≈ 19–20.

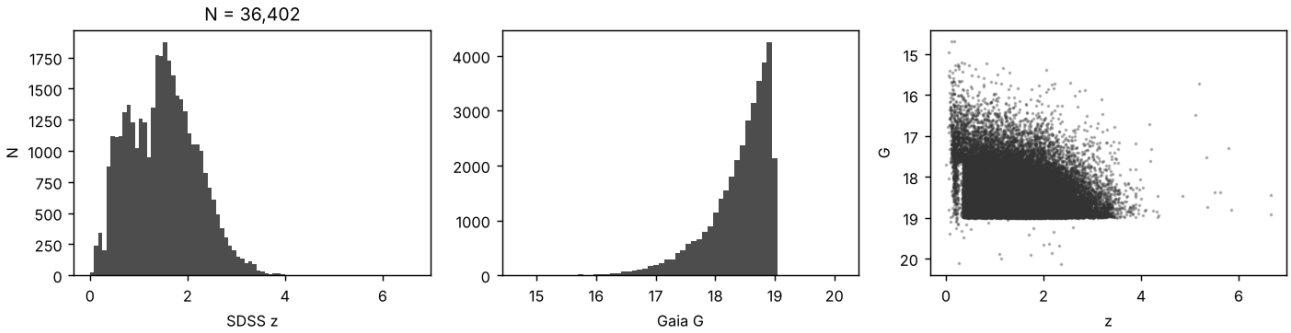


Fig. 1. Matched sample: SDSS redshift and Gaia G distributions.

2.2 XP coefficients (02_fetch_xp.py)

ESA's TAP service does not expose the XP coefficient table, and its DataLink service limits anonymous users to 50 sources per request at ~5 s of server time per source (~1,000 spectra per 25 min). The coefficients were instead retrieved from the AIP Gaia mirror (gaia.aip.de), whose TAP service has `gaiadr3.xp_continuous_mean_spectrum`: all 36,404 spectra in ~8 minutes. For 4,000 spectra fetched from both services, the coefficients are bit-identical and the errors and correlations agree to float32 rounding ($\leq 6\times10^{-8}$ relative).

3 Methods

- **Features (03_features.py).** The 110 coefficients divided by the G-band flux. They are linear in flux, so this leaves a pure shape vector. Calibrated, sampled spectra (336–1020 nm, GaiaXPy) are used only for visualisation.
- **Feature sets.** A: BP−RP, BP−G, G−RP, G. B: all 110 normalised coefficients + G. B_{low}: first 20 BP + 20 RP coefficients + G. G is included everywhere, since magnitude carries some redshift information through the luminosity function.
- **Models (05_regress.py).** kNN (k = 15, distance-weighted, standardised + 30-component whitened PCA), LightGBM (1,500 trees), MLP (256-128-64 units, quantile-to-Gaussian input scaling). Target log(1+z). 5-fold CV with a fixed seed; every prediction is out-of-fold.
- **Metrics.** $\Delta z/(1+z)$; σ_{NMAD} = 1.4826 × median absolute deviation; outlier fractions at 0.1 and 0.2; fraction within 0.01(1+z).
- **Controls.** LightGBM on shuffled redshifts; Gaia QSOC on the same objects.
- **p(z) (06_pz.py).** MLP classifier over 56 bins in log(1+z), 5-fold CV. Gives the mode, median, 68% interval, number of significant peaks and the probability integral transform (PIT).

4 Results

4.1 The signal is visible

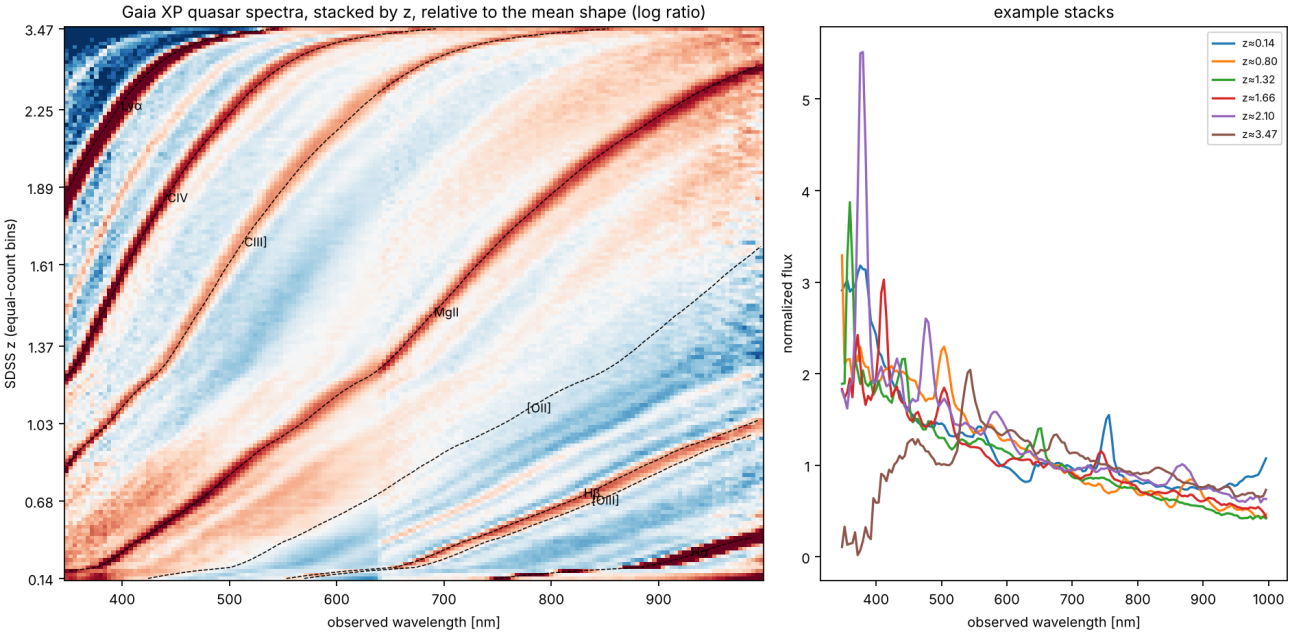


Fig. 2. Left: calibrated XP spectra stacked in equal-count redshift bins, divided by the mean observed shape (log ratio). Dashed: $\lambda_{rest}(1+z)$ tracks. Lyα, CIV, CIII], MgII, Hβ+[OIII] and Hα follow their tracks; Lyα-forest absorption shows blueward of Lyα at high z. Faint parallel stripes are partly weaker lines and partly basis-function ringing. The vertical seam near 640 nm at low z is the BP/RP join. Right: example stacks.

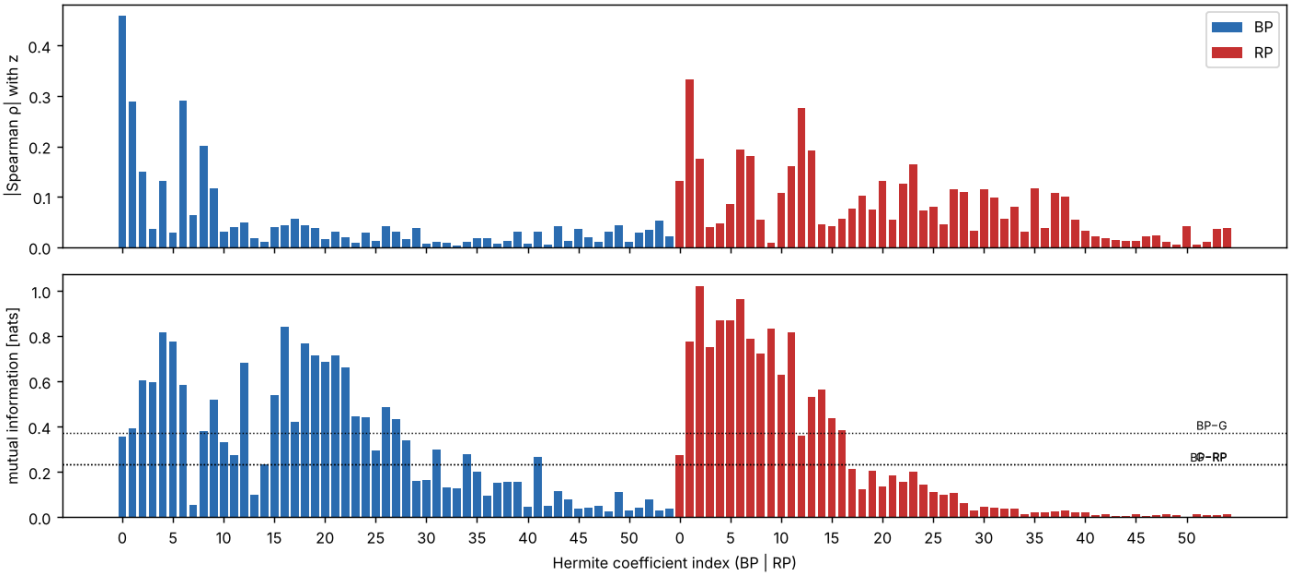


Fig. 3. |Spearman ρ| and mutual information (MI) of each coefficient with z. Dotted lines: MI of the colours (0.23, 0.37, 0.24 nats). Several single coefficients carry 0.8–1.0 nats, mostly low-order RP terms.

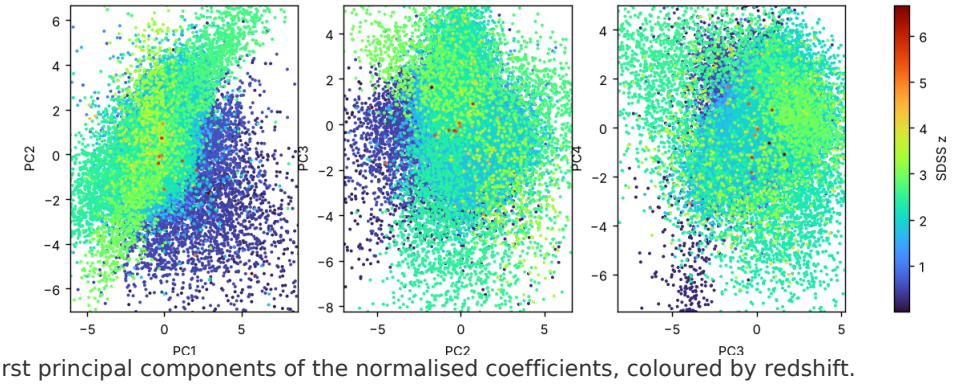


Fig. 4. First principal components of the normalised coefficients, coloured by redshift.

4.2 How much information: cross-validated regression

Features	Model	N	bias	σ_{NMAD}	outl. 0.1	outl. 0.2	within 0.01(1+z)
A_colours	kNN	36,402	+0.0141	0.1822	57.7%	30.2%	5.1%
A_colours	LightGBM	36,402	+0.0121	0.1814	57.4%	29.8%	4.9%
A_colours	MLP	36,402	+0.0146	0.1772	56.7%	28.7%	4.8%
B_xp	kNN	36,402	-0.0004	0.0107	12.6%	4.6%	54.5%
B_xp	LightGBM	36,402	+0.0008	0.0221	6.4%	1.8%	36.7%
B_xp	MLP	36,402	-0.0014	0.0099	1.5%	0.6%	66.1%
B_xp_low	kNN	36,402	+0.0001	0.0040	2.6%	1.2%	87.9%
B_xp_low	LightGBM	36,402	+0.0005	0.0202	5.7%	1.7%	40.0%
B_xp_low	MLP	36,402	+0.0017	0.0094	1.1%	0.5%	67.2%
B_xp (shuffled z control)	LightGBM	36,402	-0.0401	0.2886	75.2%	51.8%	2.5%
C_gaia_QSOC	Gaia DR3 QSOC	36,355	-0.0002	0.0030	1.5%	1.4%	97.0%
A_colours (QSOC subset)	LightGBM	36,355	+0.0121	0.1814	57.4%	29.8%	4.9%
B_xp (QSOC subset)	LightGBM	36,355	+0.0008	0.0221	6.4%	1.8%	36.8%

Table 1. 5-fold cross-validated metrics (data/metrics.csv). The last two rows rerun the models on the 36,355 objects that also have a QSOC redshift.

- Coefficients beat colours by a factor 20–40 in σ_{NMAD} and up to ~60 in catastrophic outliers. Colours cannot separate redshifts between z ≈ 1 and 2.5: predictions there pile up near 1.7.
- kNN on the low-order coefficients is the sharpest (σ_{NMAD} 0.0040), close to QSOC. Dropping the noisy high-order coefficients matters for kNN, much less for trees.
- The MLPs give the fewest catastrophic outliers (0.50–0.59%), about 3× fewer than QSOC. Plain standardisation broke the MLP (on 3,000 objects it was no better than colours); quantile scaling fixed it.
- The shuffled-redshift control has no skill (σ_{NMAD} 0.29, 52% outliers).
- With 3,000 objects (an early random subset), the scores were much worse (kNN on all coefficients σ_{NMAD} 0.074 vs 0.011 with 36k): these models are data-limited.

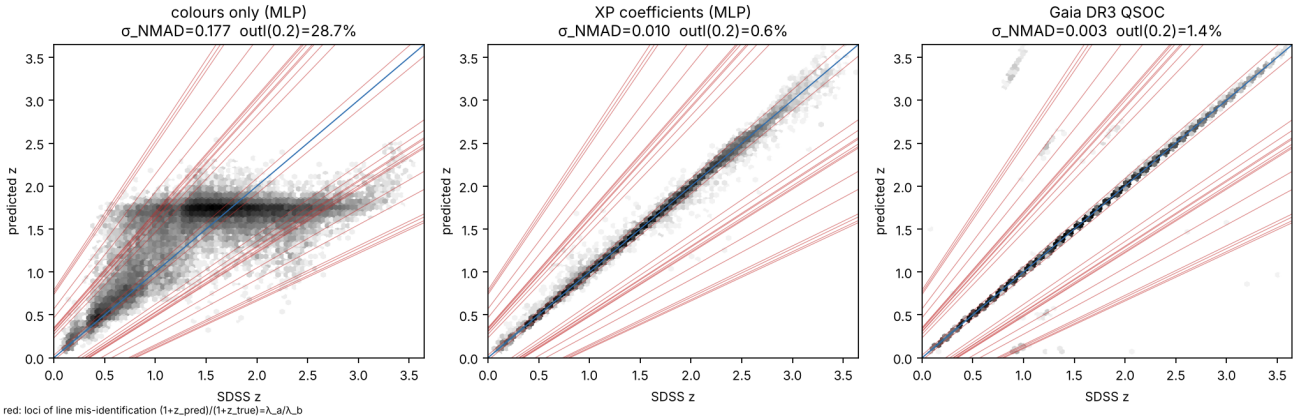


Fig. 5. Predicted vs SDSS redshift: colours only, XP coefficients (MLP), Gaia QSOC. Red lines: loci where one emission line is mistaken for another, $(1+z_{pred})/(1+z_{true}) = \lambda_a/\lambda_b$. QSOC's remaining outliers include a cluster of z ≈ 0.5–1 quasars placed near z ≈ 3.5.

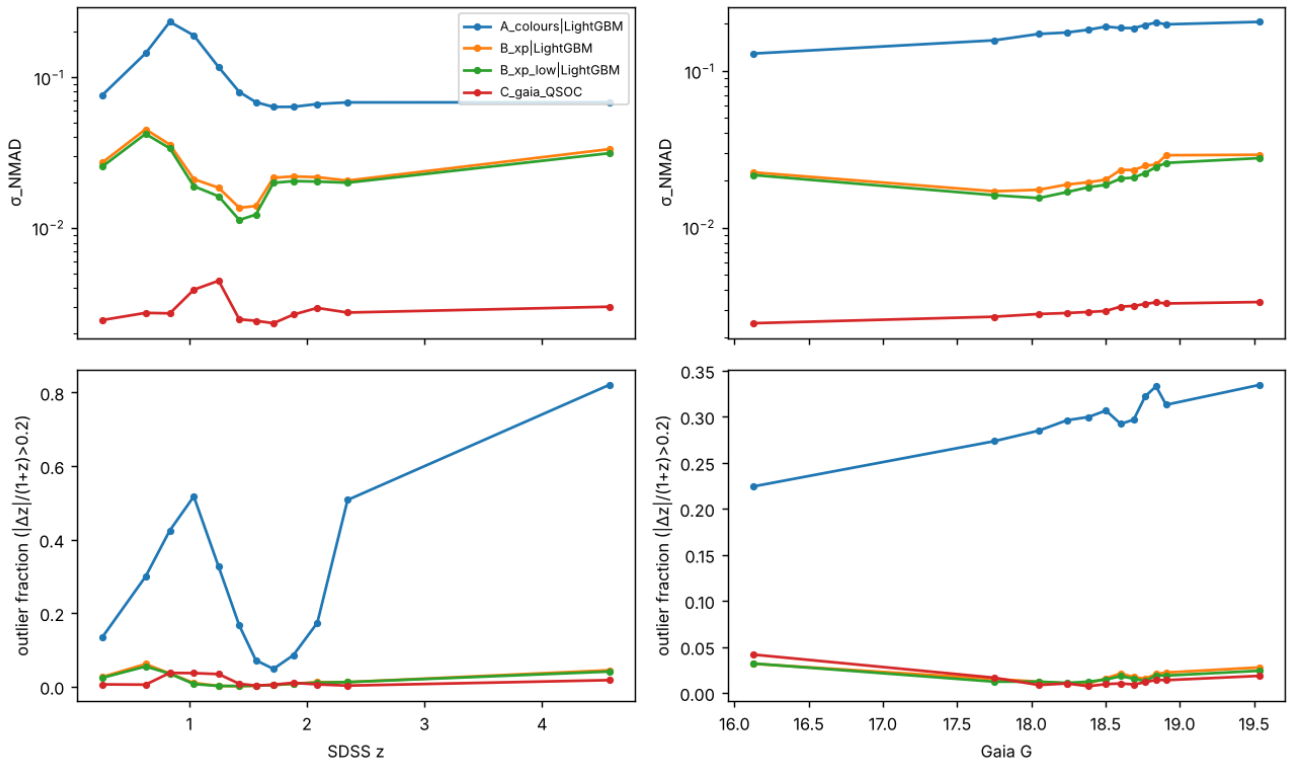


Fig. 6. σ_{NMAD} and outlier fraction vs SDSS redshift and Gaia G (LightGBM models and QSOC). All methods degrade smoothly towards G ≈ 19.5.

4.3 Redshift probabilities and line confusion

The p(z) classifier gives 0.63% outliers (mode) and σ_{NMAD} 0.011 (median). 2.3% of objects get a p(z) with more than one significant peak. For 43% of the remaining catastrophic outliers, the model still puts more than 20% probability within 0.05(1+z) of the true redshift, so most of its failures are flagged ambiguities. The 68% intervals are too wide (81% coverage), probably because the bins are coarse. A first attempt with a LightGBM multiclass model diverged (one-hot probabilities, 33% outliers) and was replaced.

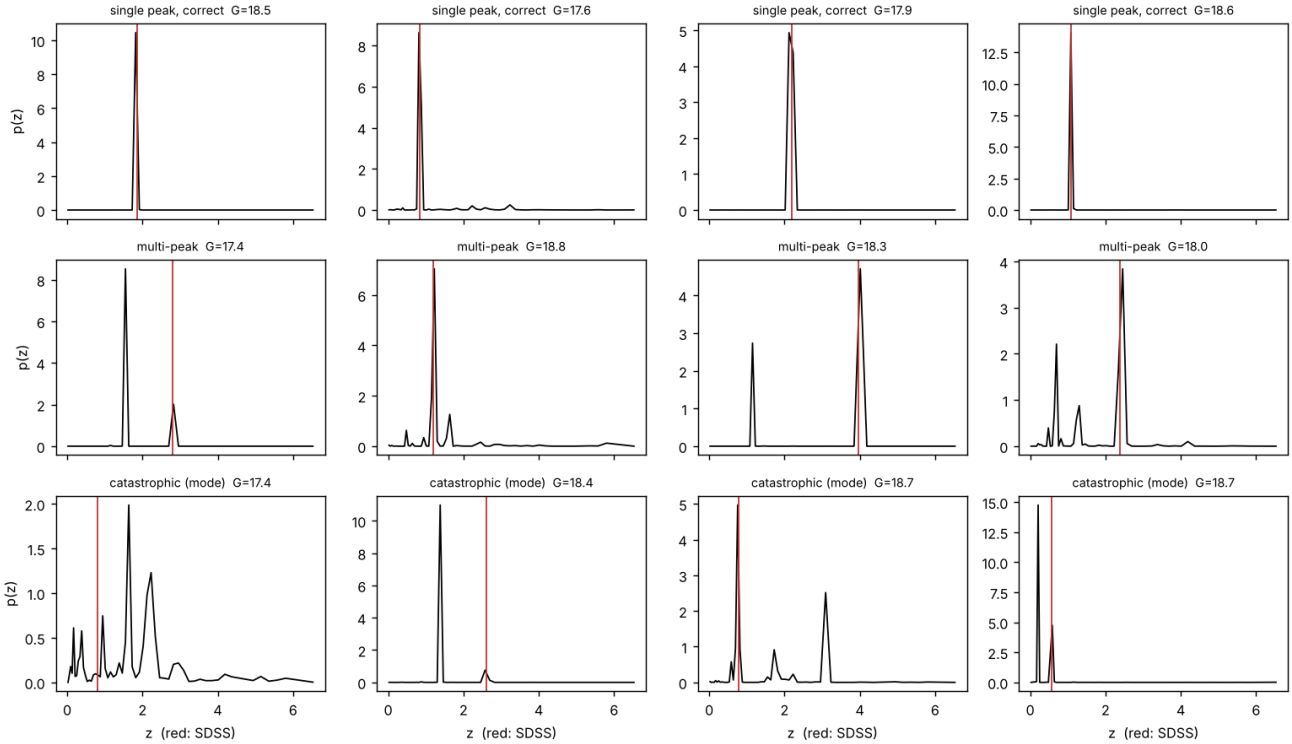


Fig. 7. Example p(z): single-peaked correct, multi-peaked, and catastrophic cases. Red: SDSS redshift.

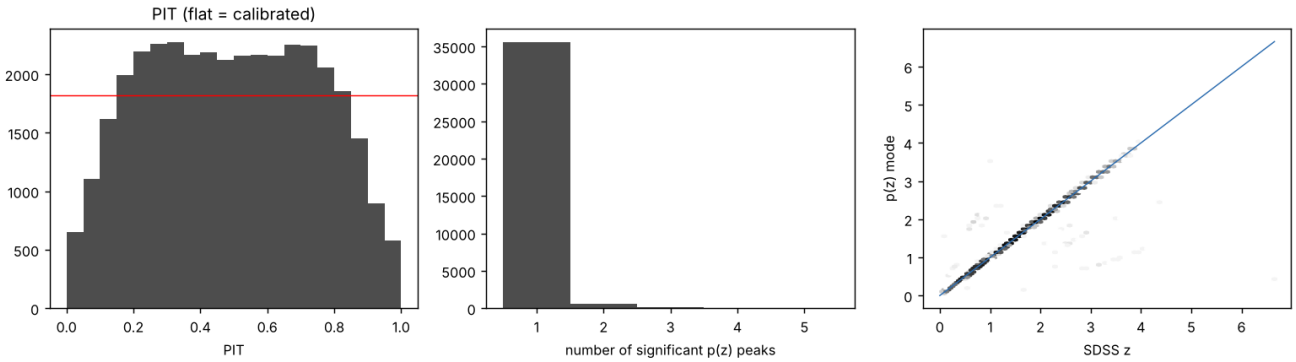


Fig. 8. PIT histogram, number of significant peaks, and p(z) mode vs SDSS z.

5 Caveats

- The sample is DR16Q-selected and relatively bright ($G \lesssim 19.5$). Its performance does not carry over to fainter or differently selected quasars, or to contamination by stars and galaxies, which was not modelled here.
- Cross-validation is within DR16Q only. No independent test sample (e.g. DESI) was used, and no spatial-block split was tried.
- QSOC's template was built from SDSS DR12Q quasars, so the QSOC comparison is not fully independent of SDSS. The comparison with published deep-learning redshifts was not done (Sect. 6).
- Hyperparameters were not tuned. The p(z) intervals are not calibrated.

6 Prior work and assessment

A literature search found refereed work that already estimates quasar redshifts from Gaia XP spectra with models trained on SDSS. Only abstracts and summaries were read, because the full texts were not reachable from the analysis environment.

- Delchambre et al. 2023, A&A 674, A31 — Gaia DR3 QSOC: χ^2 fit of a composite quasar template (from SDSS DR12Q) to BP/RP spectra.
- Storey-Fisher et al. 2024, ApJ — Quaia: kNN on QSOC redshifts plus Gaia/unWISE photometry, trained on SDSS; reports outliers from line aliasing.
- Fu et al. 2024, ApJS 271, 54 (CatNorth) and Fu et al. 2025, ApJS 279, 54 (CatSouth) — a CNN on sampled XP spectra, trained on SDSS (CatSouth reports $\sigma_{\mathrm{NMAD}} \approx 0.030$, outliers $\approx 2.5\%$).
- “Unveiling Quasars in Gaia DR3: Multimodal Classification and Redshift Estimation of BP/RP Sources with Deep Learning”, ApJS, doi:10.3847/1538-4365/ae4003 — a deep network on the Hermite coefficients, sampled fluxes and catalogue features (SpecNet-Z); catalogue on Zenodo (10.5281/zenodo.17960118).

The core idea is therefore established. The only potentially new point, that a simple kNN on low-order coefficients may be much sharper than the published CNNs (0.004 vs ≈ 0.03), would need a comparison on identical objects. That comparison must account for the published models having been trained on SDSS quasars that overlap this sample. It was not pursued.

7 Reproducibility

All code is in `gaia_qso_z/` and listed in full in the Appendix: `common.py`, `01_crossmatch.py` … `06_pz.py`, `run_all.sh`, `make_report.py` (this report), `requirements.txt` and `README.md`. To reproduce: `pip install -r requirements.txt` then `./run_all.sh`. Network access is needed to data.sdss.org (DR16Q, 1.7 GB), gea.esac.esa.int (Gaia TAP) and gaia.aip.de (XP coefficients). The full run takes about an hour on 4 CPUs, dominated by the cross-match (~10 min) and model training. Raw data products (~3 GB) are not part of the code and are regenerated by the scripts.

Python 3.13 with: numpy==2.5.3, pandas==3.0.5, scipy==1.18.1, astropy==8.0.2, astroquery==0.4.11, pyvo==1.10.0, requests==2.34.2, GaiaXPy==2.1.5, scikit-learn==1.9.1, lightgbm==4.7.0, matplotlib==3.11.2, pyarrow==26.0.1.

Appendix: source code

README.md

````
# Quasar redshifts from Gaia DR3 XP coefficients

Do the Gaia DR3 BP/RP (XP) Hermite coefficients carry quasar redshift information?
SDSS DR16Q spectroscopic redshifts are matched to Gaia DR3 sources with published XP
coefficients (36,402 quasars), and redshift is predicted from the coefficients with
cross-validated kNN, LightGBM and MLP models, compared with Gaia colours and Gaia's QSOC.

## Run

```bash
pip install -r requirements.txt
./run_all.sh            # or run 01_ ... 06_ in order
```

GaiaXPy 2.1.5 depends on `hdfs`/`docopt`, whose source builds can fail on Python 3.13; if so,
install `docopt-ng` and copy the pure-Python `hdfs` package from its sdist into site-packages.

Network hosts needed: `data.sdss.org` (DR16Q, 1.7 GB), `gea.esac.esa.int` (Gaia TAP),
`gaia.aip.de` (XP coefficients via TAP; ~8 min for all spectra). PyPI for installs.

## Scripts

| File | Step |
|---|---|
| `common.py` | paths, emission-line list, metrics, TAP upload helper (pyvo) |
| `01_crossmatch.py` | DR16Q → Gaia DR3 via DR16Q's DR2 ids and `gaiadr3.dr2_neighbourhood`; QSOC redshifts |
| `02_fetch_xp.py` | XP coefficients from the AIP mirror (default) or ESA DataLink (`--source esa`, slow) |
| `03_features.py` | flux-normalised coefficients; GaiaXPy-calibrated spectra |
| `04_explore.py` | redshift ladder, coefficient–z mutual information, PCA |
| `05_regress.py` | 5-fold CV regression: colours vs coefficients vs QSOC; shuffled-z control |
| `06_pz.py` | p(z) via MLP classifier over log(1+z) bins |

Outputs go to `data/` and `figures/`. `REPORT.pdf` summarises the results.
````

requirements.txt

```
numpy==2.5.3
pandas==3.0.5
scipy==1.18.1
astropy==8.0.2
astroquery==0.4.11
pyvo==1.10.0
requests==2.34.2
GaiaXPy==2.1.5
scikit-learn==1.9.1
lightgbm==4.7.0
matplotlib==3.11.2
pyarrow==26.0.1
```

run_all.sh

```
#!/usr/bin/env bash
# Full pipeline. Steps 1-2 need network access to data.sdss.org, gea.esac.esa.int and gaia.aip.de.
set -euo pipefail
cd "$(dirname "$0")"
for s in 01_crossmatch 02_fetch_xp 03_features 04_explore 05_regress 06_pz; do
  echo "=== $s ==="; python3 "$s.py" 2>&1 | tee "data/log_$s.txt"
done
```

common.py

```
"""Shared paths, constants and helpers for the Gaia XP x SDSS quasar redshift study."""
import os
from pathlib import Path

import numpy as np

ROOT = Path(__file__).resolve().parent
DATA = Path(os.environ.get("GQZ_DATA", ROOT / "data"))
FIG = Path(os.environ.get("GQZ_FIG", ROOT / "figures"))
DATA.mkdir(parents=True, exist_ok=True)
FIG.mkdir(parents=True, exist_ok=True)

DR16Q_URL = "https://data.sdss.org/sas/dr16/eboss/qso/DR16Q/DR16Q_v4.fits"
DR16Q_FITS = DATA / "DR16Q_v4.fits"
DR16Q_SLIM = DATA / "dr16q_slim.parquet"
MATCH = DATA / "dr16q_gaia_xp_match.parquet"
XP_RAW = DATA / "xp_continuous.pkl"          # list/array columns -> pickle keeps dtypes intact
FEATURES = DATA / "features.parquet"
SPECTRA = DATA / "calibrated_spectra.npz"
PRED = DATA / "cv_predictions.parquet"

SEED = 42
N_COEF = 55

# Broad emission lines in the rest frame (nm)
```

```
LINES = {
    "Lyα": 121.6,
    "CIV": 154.9,
    "CIII]": 190.9,
    "MgII": 279.8,
    "[OII]": 372.7,
    "Hβ": 486.1,
    "[OIII]": 500.7,
    "Hα": 656.3,
}
XP_RANGE = (336.0, 1020.0)  # nm, GaiaXPy calibrated range


def metrics(z_true, z_pred):
    """Standard photo-z metrics on dz = (z_pred - z_true) / (1 + z_true)."""
    z_true = np.asarray(z_true, float)
    z_pred = np.asarray(z_pred, float)
    dz = (z_pred - z_true) / (1.0 + z_true)
    med = np.median(dz)
    return {
        "N": int(dz.size),
        "bias_median": float(med),
        "sigma_NMAD": float(1.4826 * np.median(np.abs(dz - med))),
        "outlier_0.1": float(np.mean(np.abs(dz) > 0.1)),
        "outlier_0.2": float(np.mean(np.abs(dz) > 0.2)),
        "frac_|dz|<0.01": float(np.mean(np.abs(z_pred - z_true) < 0.01 * (1 + z_true))),
    }


def confusion_ratios():
    """(1+z_pred)/(1+z_true) ratios produced by mistaking line i for line j."""
    names = list(LINES)
    out = []
    for i, a in enumerate(names):
        for b in names[i + 1:]:
            r = LINES[b] / LINES[a]
            # observed λ = λ_a(1+z_true) = λ_b(1+z_pred)  =>  (1+z_pred)/(1+z_true) = λ_a/λ_b
            out.append((f"{a} read as {b}", 1 / r))
            out.append((f"{b} read as {a}", r))
    return out


GAIA_TAP = "https://gea.esac.esa.int/tap-server/tap"
AIP_TAP = "https://gaia.aip.de/tap"   # AIP mirror: also exposes gaiadr3.xp_continuous_mean_spectrum


def gaia_upload_query(query, upload, name, retries=4, poll=5, max_wait=3 * 3600, url=GAIA_TAP):
    """Run an async ADQL query on a Gaia TAP service (ESA by default) with an uploaded astropy Table.

    Uses pyvo (requests honours the HTTPS proxy; astroquery's Gaia client does not).
    Retries on network errors and archive-side aborts.
    """
    import time

    import pyvo

    tap = pyvo.dal.TAPService(url)
    last = None
    for attempt in range(retries):
        job = None
        try:
            job = tap.submit_job(query, uploads={name: upload})
            job.run()
            t0 = time.time()
            while True:
                time.sleep(poll)
                phase = job.phase
                if phase in ("COMPLETED", "ERROR", "ABORTED"):
                    break
                if time.time() - t0 > max_wait:
                    raise TimeoutError("job did not finish")
            if phase != "COMPLETED":
                try:
                    job.raise_if_error()
                except Exception as err:
                    raise RuntimeError(f"job {phase}: {err}") from None
                raise RuntimeError(f"job {phase}")
            res = job.fetch_result().to_table()
            try:
                job.delete()
            except Exception:
                pass
            return res
        except Exception as e:
            last = e
            wait = 2 ** (attempt + 2)
            print(f"  attempt {attempt + 1} failed ({str(e)[:160]}); retry in {wait}s", flush=True)
            time.sleep(wait)
    raise RuntimeError(f"query failed after {retries} attempts: {last}")
```

01_crossmatch.py

```
"""Step 1: SDSS DR16Q x Gaia DR3 match.

DR16Q already carries its Gaia DR2 counterpart (GAIA_DESIGNATION, matched by the SDSS team).
```

```
DR2 ids are uploaded to the Gaia archive and mapped to DR3 through gaiadr3.dr2_neighbourhood,
joined to gaia_source. This id-based join is ~100x faster than a positional join, which the
archive aborts after 60 s for anonymous users. Per quasar we keep the DR3 neighbour closest
to the SDSS position, requiring < 1 arcsec.

A positional 1" cross-match of the first ~180k DR16Q rows (data/xmatch_batches/, from the earlier
positional run) is used to measure how complete the id route is for sources with XP spectra.

Outputs:
  data/dr16q_slim.parquet           slim DR16Q (one row per quasar, qid = row index)
  data/dr16q_gaia_xp_match.parquet  Gaia DR3 counterpart per quasar, has_xp_continuous, QSOC z
"""
import shutil
import time
import urllib.request

import numpy as np
import pandas as pd
from astropy.io import fits
from astropy.table import Table

from common import DATA, DR16Q_FITS, DR16Q_SLIM, DR16Q_URL, MATCH, gaia_upload_query

BATCH = 20_000
G_DR2_MAX = 20.5          # XP spectra exist down to G ~ 20; skip fainter DR2 counterparts
RADIUS_ARCSEC = 1.0
ID_DIR = DATA / "idmatch_batches"
ID_DIR.mkdir(exist_ok=True)
POS_DIR = DATA / "xmatch_batches"

WANTED = ["SDSS_NAME", "RA", "DEC", "Z", "SOURCE_Z", "ZWARNING", "Z_PIPE", "BAL_PROB", "IS_QSO_FINAL",
          "GAIA_G_MAG"]

QUERY = """
SELECT u.qid, n.dr2_source_id, n.dr3_source_id AS source_id, n.angular_distance AS dr2_dr3_dist_mas,
       DISTANCE(POINT('ICRS', u.ra, u.dec), POINT('ICRS', g.ra, g.dec)) * 3600 AS sep_arcsec,
       g.phot_g_mean_mag, g.phot_bp_mean_mag, g.phot_rp_mean_mag,
       g.phot_g_mean_flux, g.phot_bp_mean_flux, g.phot_rp_mean_flux,
       g.phot_bp_rp_excess_factor, g.has_xp_continuous, g.classprob_dsc_combmod_quasar,
       g.parallax, g.parallax_over_error, g.pm
FROM tap_upload.q AS u
JOIN gaiadr3.dr2_neighbourhood AS n ON n.dr2_source_id = u.dr2_source_id
JOIN gaiadr3.gaia_source AS g ON g.source_id = n.dr3_source_id
"""
QSOC_QUERY = """
SELECT q.source_id, q.redshift_qsoc, q.redshift_qsoc_lower, q.redshift_qsoc_upper, q.zscore_qsoc, q.flags_qsoc
FROM tap_upload.ids AS u JOIN gaiadr3.qso_candidates AS q ON q.source_id = u.source_id
"""


def download_dr16q():
    if DR16Q_FITS.exists():
        return
    print(f"Downloading {DR16Q_URL} ...", flush=True)
    tmp = DR16Q_FITS.with_suffix(".part")
    with urllib.request.urlopen(DR16Q_URL) as r, open(tmp, "wb") as f:
        shutil.copyfileobj(r, f, length=1 << 22)
    tmp.rename(DR16Q_FITS)


def native(v):
    return v.byteswap().view(v.dtype.newbyteorder()) if v.dtype.byteorder == ">" else v


def slim_dr16q():
    if DR16Q_SLIM.exists():
        df = pd.read_parquet(DR16Q_SLIM)
        if "dr2_source_id" in df:
            return df
    with fits.open(DR16Q_FITS, memmap=True) as h:
        d = h[1].data
        names = set(d.columns.names)
        cols = {c.lower(): native(np.asarray(d[c])) for c in WANTED if c in names}
        if "PSFMAG" in names:  # ugriz PSF magnitudes
            psf = native(np.asarray(d["PSFMAG"]))
            for i, b in enumerate("ugriz"):
                cols[f"psfmag_{b}"] = psf[:, i]
        des = np.char.strip(np.asarray(d["GAIA_DESIGNATION"]).astype(str))
    df = pd.DataFrame(cols).rename(columns={"gaia_g_mag": "g_dr2"})
    df["sdss_name"] = df["sdss_name"].astype(str).str.strip()
    df["dr2_source_id"] = np.array([int(s.split()[-1]) if s.startswith("Gaia") else -1 for s in des], dtype=np.int64)
    df.insert(0, "qid", np.arange(len(df), dtype=np.int64))
    df.to_parquet(DR16Q_SLIM)
    return df


def run_batches(q):
    up = q[(q.dr2_source_id > 0) & (q.g_dr2 < G_DR2_MAX)][["qid", "ra", "dec", "dr2_source_id"]]
    print(f"uploading {len(up):,} DR2 ids (G_DR2 < {G_DR2_MAX})")
    for start in range(0, len(up), BATCH):
        out = ID_DIR / f"ids_{start:07d}.parquet"
        if out.exists():
            continue
        t0 = time.time()
        res = gaia_upload_query(QUERY, Table.from_pandas(up.iloc[start:start + BATCH]), "q", retries=6)
        res.to_pandas().to_parquet(out)
```

```
        print(f"  {min(start + BATCH, len(up)):>7,}/{len(up):,}: {len(res)} rows ({time.time() - t0:.0f}s)", flush=True)
    return pd.concat([pd.read_parquet(p) for p in sorted(ID_DIR.glob("ids_*.parquet"))], ignore_index=True)


def as_bool(s):
    return s.astype(str).str.lower().isin(["true", "1"])


def validate(m):
    """Compare with the positional cross-match where it exists (first ~180k DR16Q rows)."""
    files = sorted(POS_DIR.glob("batch_q*.parquet"))
    if not files:
        return
    pos = pd.concat([pd.read_parquet(p) for p in files], ignore_index=True)
    covered = set()
    for p in files:                         # file names: batch_q<start>_<stop>.parquet
        a, b = (int(x) for x in p.stem.removeprefix("batch_q").split("_"))
        covered.update(range(a, b))
    pos = pos[as_bool(pos.has_xp_continuous)].sort_values("sep_arcsec").drop_duplicates("qid")
    idm = m[m.qid.isin(covered) & m.has_xp_continuous]
    both = pos.merge(idm[["qid", "source_id"]], on="qid", how="left", suffixes=("", "_id"))
    rec = both.source_id_id.notna().mean()
    same = (both.source_id == both.source_id_id)[both.source_id_id.notna()].mean()
    extra = (~idm.qid.isin(pos.qid)).sum()
    print(f"\nValidation on {len(covered):,} DR16Q rows that also have a positional match run:")
    print(f"  positional XP matches: {len(pos):,}; recovered by id route: {rec:.1%} (same DR3 source: {same:.1%});"
          f" id-route XP matches not in positional set: {extra}")
    lost = both[both.source_id_id.isna()]
    if len(lost):
        print("  G of missed sources, percentiles 10/50/90:", np.percentile(lost.phot_g_mean_mag, [10, 50, 90]).round(2))


def main():
    download_dr16q()
    q = slim_dr16q()
    print(f"DR16Q quasars: {len(q):,}; with a DR2 counterpart: {(q.dr2_source_id > 0).sum():,}")
    m = run_batches(q)
    m["has_xp_continuous"] = as_bool(m["has_xp_continuous"])
    m = m[m.sep_arcsec < RADIUS_ARCSEC]
    m = m.sort_values("sep_arcsec").drop_duplicates("qid", keep="first")
    m = m.sort_values("sep_arcsec").drop_duplicates("source_id", keep="first")
    m = m.merge(q.drop(columns=["dr2_source_id"]), on="qid", how="left")

    ids = m.loc[m.has_xp_continuous, "source_id"].astype(np.int64).to_numpy()
    qs = pd.concat([gaia_upload_query(QSOC_QUERY, Table({"source_id": ids[i:i + BATCH]}), "ids", retries=6).to_pandas()
                    for i in range(0, len(ids), BATCH)], ignore_index=True)
    m = m.merge(qs, on="source_id", how="left")
    m.to_parquet(MATCH)

    xp = m[m.has_xp_continuous]
    print("\n=== Cross-match summary ===")
    print(f"DR16Q quasars                 {len(q):>9,}")
    print(f"with a Gaia DR3 source <1\"    {len(m):>9,}  ({len(m) / len(q):.1%})  [G_DR2 < {G_DR2_MAX}]")
    print(f"... of which have XP coeffs   {len(xp):>9,}  ({len(xp) / len(q):.2%} of DR16Q)")
    print(f"... with Gaia QSOC redshift   {xp.redshift_qsoc.notna().sum():>9,}")
    print(f"median separation (XP set)    {xp.sep_arcsec.median():.3f}\"")
    print("XP-set G mag percentiles 5/50/95:", np.percentile(xp.phot_g_mean_mag.dropna(), [5, 50, 95]).round(2))
    print("XP-set z percentiles 5/50/95:   ", np.percentile(xp.z, [5, 50, 95]).round(2))
    validate(m)


if __name__ == "__main__":
    main()
```

02_fetch_xp.py

```
"""Step 2: fetch Gaia DR3 XP continuous coefficients (BP+RP, 55 each) for matched quasars.

Default source: the AIP Gaia mirror (gaia.aip.de), whose TAP service exposes
gaiadr3.xp_continuous_mean_spectrum, so an uploaded-id join fetches 2,000 spectra in seconds.
Fallback (--source esa): ESA's TAP does not expose that table, so source_id batches are POSTed to
the ESA DataLink service (50 ids per anonymous request, ~5 s per source: very slow).
Output: data/xp_continuous.pkl (archive column names, array columns as numpy arrays;
        this is the input format GaiaXPy expects).
"""
import io
import sys
import time
import zipfile
from concurrent.futures import ThreadPoolExecutor

import numpy as np
import pandas as pd
import requests
from astropy.table import Table

from common import AIP_TAP, DATA, MATCH, XP_RAW, gaia_upload_query

BATCH = 1_000           # sources per saved file
PER_REQUEST = 50        # DataLink limit for anonymous users
THREADS = 4
DATALINK = "https://gea.esac.esa.int/data-server/data"
BATCH_DIR = DATA / "xp_batches"
BATCH_DIR.mkdir(exist_ok=True)
AIP_BATCH = 2_000
```

```
AIP_QUERY = """
SELECT x.* FROM TAP_UPLOAD.ids AS u
JOIN gaiadr3.xp_continuous_mean_spectrum AS x ON x.source_id = u.source_id
"""



def fetch_request(ids, retries=6):
    payload = {"RETRIEVAL_TYPE": "XP_CONTINUOUS", "DATA_STRUCTURE": "DATAMODEL_GAIA", "RELEASE": "Gaia DR3",
               "FORMAT": "votable", "ID": ",".join(map(str, ids))}
    for attempt in range(retries):
        try:
            r = requests.post(DATALINK, data=payload, timeout=(30, 600))
            r.raise_for_status()
            blob = r.content
            if blob[:2] == b"PK":   # zipped multi-file answer
                with zipfile.ZipFile(io.BytesIO(blob)) as z:
                    tabs = [Table.read(io.BytesIO(z.read(n)), format="votable") for n in z.namelist()]
                from astropy.table import vstack
                return vstack(tabs)
            return Table.read(io.BytesIO(blob), format="votable")
        except Exception as e:
            wait = 2 ** (attempt + 2)
            print(f"  attempt {attempt + 1} failed ({str(e)[:150]}); retry in {wait}s", flush=True)
            time.sleep(wait)
    raise RuntimeError("DataLink batch failed repeatedly")
ARRAY_COLS = [f"{b}_{c}" for b in ("bp", "rp")
              for c in ("coefficients", "coefficient_errors", "coefficient_correlations")]


def to_numpy_arrays(df):
    for c in ARRAY_COLS:
        if c in df:
            df[c] = [np.asarray(np.ma.filled(v, np.nan) if np.ma.isMaskedArray(v) else v, dtype=float)
                     for v in df[c]]
    return df


def fetch_aip(ids):
    n = int(np.ceil(len(ids) / AIP_BATCH))
    for i in range(n):
        out = BATCH_DIR / f"aip_{i:03d}.pkl"
        if out.exists():
            continue
        t0 = time.time()
        tab = gaia_upload_query(AIP_QUERY, Table({"source_id": ids[i * AIP_BATCH:(i + 1) * AIP_BATCH]}), "ids",
                                retries=6, poll=2, url=AIP_TAP)
        df = pd.DataFrame({c: list(tab[c]) if tab[c].ndim > 1 or tab[c].dtype == object else np.asarray(tab[c])
                           for c in tab.colnames})
        to_numpy_arrays(df).to_pickle(out)
        print(f"AIP batch {i + 1}/{n}: {len(df)} spectra ({time.time() - t0:.0f}s)", flush=True)
    return sorted(BATCH_DIR.glob("aip_*.pkl"))


def main():
    m = pd.read_parquet(MATCH)
    ids = m.loc[m.has_xp_continuous, "source_id"].astype(np.int64).to_numpy()
    # random order: every completed batch file is a random subsample, so the analysis can start
    # on a partial download (the archive serves ~5 s of CPU per source to anonymous users)
    ids = np.random.default_rng(0).permutation(np.sort(ids))
    print(f"Sources with XP: {len(ids):,}")
    n_batches = int(np.ceil(len(ids) / BATCH))
    if "esa" not in sys.argv[1:] or "--source" not in sys.argv:
        files = fetch_aip(ids)
        xp = pd.concat([pd.read_pickle(p) for p in files], ignore_index=True).drop_duplicates("source_id")
        print(f"XP spectra retrieved: {len(xp):,}  (of {len(ids):,} with XP)")
        print("bp/rp coefficient vector lengths:", {len(v) for v in xp.bp_coefficients}, {len(v) for v in xp.rp_coefficients})
        xp.to_pickle(XP_RAW)
        return
    assemble_only = "--assemble" in sys.argv
    for i in range(n_batches):
        if assemble_only:
            break
        out = BATCH_DIR / f"xp_{i:03d}.pkl"
        if out.exists():
            continue
        t0 = time.time()
        chunk = ids[i * BATCH:(i + 1) * BATCH]
        with ThreadPoolExecutor(THREADS) as ex:
            parts = list(ex.map(fetch_request, [chunk[j:j + PER_REQUEST] for j in range(0, len(chunk), PER_REQUEST)]))
        from astropy.table import vstack
        tab = vstack([t for t in parts if len(t)])
        df = pd.DataFrame({c: list(tab[c]) if tab[c].ndim > 1 or tab[c].dtype == object else np.asarray(tab[c])
                           for c in tab.colnames})
        to_numpy_arrays(df).to_pickle(out)
        print(f"batch {i + 1}/{n_batches}: {len(df)} spectra ({time.time() - t0:.0f}s)", flush=True)

    xp = pd.concat([pd.read_pickle(p) for p in sorted(BATCH_DIR.glob("xp_*.pkl"))], ignore_index=True)
    xp = xp.drop_duplicates("source_id")
    print(f"XP spectra retrieved: {len(xp):,}  (of {len(ids):,} with XP)")
    print("bp/rp coefficient vector lengths:", {len(v) for v in xp.bp_coefficients}, {len(v) for v in xp.rp_coefficients})
    xp.to_pickle(XP_RAW)


if __name__ == "__main__":
    main()
```

03_features.py

```
"""Step 3: build model features and calibrated spectra.

Features (one row per quasar):
  - c_bp_00..54, c_rp_00..54 : XP coefficients divided by the G-band flux (brightness removed;
                               the coefficients are linear in flux, so this is a pure shape vector)
  - e_bp_.., e_rp_..         : matching scaled 1-sigma errors
  - colours / magnitudes, DR16Q z, Gaia QSOC z
Calibrated spectra (GaiaXPy.calibrate) on 336-1020 nm are saved for the interpretable plots.
"""
import warnings

import numpy as np
import pandas as pd
from gaiaxpy import calibrate

from common import FEATURES, MATCH, N_COEF, SPECTRA, XP_RAW, XP_RANGE

SAMPLING = np.arange(XP_RANGE[0], XP_RANGE[1] + 0.1, 4.0)
CAL_BATCH = 2_000


def coef_matrix(xp, band, what="coefficients"):
    return np.vstack([np.asarray(v, float)[:N_COEF] for v in xp[f"{band}_{what}"]])


def main():
    m = pd.read_parquet(MATCH)
    xp = pd.read_pickle(XP_RAW)
    xp = xp[xp.source_id.isin(m.source_id)].reset_index(drop=True)
    m = m.set_index("source_id").loc[xp.source_id].reset_index()

    gflux = m.phot_g_mean_flux.to_numpy()
    feats = {}
    for band in ("bp", "rp"):
        C = coef_matrix(xp, band) / gflux[:, None]
        E = coef_matrix(xp, band, "coefficient_errors") / gflux[:, None]
        for k in range(N_COEF):
            feats[f"c_{band}_{k:02d}"] = C[:, k]
            feats[f"e_{band}_{k:02d}"] = E[:, k]
        feats[f"{band}_n_relevant_bases"] = xp[f"{band}_n_relevant_bases"].to_numpy()
        feats[f"{band}_n_measurements"] = xp[f"{band}_n_measurements"].to_numpy()
    f = pd.DataFrame(feats)
    keep = ["source_id", "qid", "z", "zwarning", "source_z", "bal_prob", "sep_arcsec",
            "phot_g_mean_mag", "phot_bp_mean_mag", "phot_rp_mean_mag", "phot_bp_rp_excess_factor",
            "redshift_qsoc", "redshift_qsoc_lower", "redshift_qsoc_upper", "flags_qsoc",
            "psfmag_u", "psfmag_g", "psfmag_r", "psfmag_i", "psfmag_z"]
    meta = m[[c for c in keep if c in m]].reset_index(drop=True)
    meta["bp_rp"] = meta.phot_bp_mean_mag - meta.phot_rp_mean_mag
    meta["bp_g"] = meta.phot_bp_mean_mag - meta.phot_g_mean_mag
    meta["g_rp"] = meta.phot_g_mean_mag - meta.phot_rp_mean_mag
    df = pd.concat([meta, f], axis=1)

    good = np.isfinite(df.z) & (df.z > 0) & np.isfinite(df[[c for c in df if c.startswith("c_")]]).all(axis=1)
    good &= np.isfinite(df.bp_rp)
    print(f"rows: {len(df):,}, usable: {good.sum():,}  (dropped {(~good).sum()} with bad z / NaN coeffs / no BP-RP)")
    df = df[good].reset_index(drop=True)
    xp = xp.set_index("source_id").loc[df.source_id].reset_index()
    df.to_parquet(FEATURES)

    # ---- calibrated, sampled spectra for interpretation ----
    cal_cols = ["source_id"] + [f"{b}_{c}" for b in ("bp", "rp") for c in (
        "n_parameters", "standard_deviation", "coefficients", "coefficient_errors", "coefficient_correlations",
        "basis_function_id", "n_relevant_bases", "relative_shrinking")]
    cal_cols = [c for c in cal_cols if c in xp]
    flux, ferr = [], []
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        for i in range(0, len(xp), CAL_BATCH):
            spec, _ = calibrate(xp.iloc[i:i + CAL_BATCH][cal_cols].copy(), sampling=SAMPLING, save_file=False)
            flux.append(np.vstack(spec["flux"].to_numpy()))
            ferr.append(np.vstack(spec["flux_error"].to_numpy()))
            print(f"calibrated {min(i + CAL_BATCH, len(xp)):,}/{len(xp):,}", flush=True)
    np.savez_compressed(SPECTRA, source_id=df.source_id.to_numpy(), wl=SAMPLING,
                        flux=np.vstack(flux), flux_error=np.vstack(ferr), z=df.z.to_numpy())
    print("saved", FEATURES.name, SPECTRA.name)


if __name__ == "__main__":
    main()
```

04_explore.py

```
"""Step 4: look at the signal before fitting anything.

  figures/redshift_ladder.png   calibrated spectra stacked in z bins, relative to the mean shape,
                                with emission-line tracks lambda_rest*(1+z) overlaid
  figures/coef_vs_z.png         |Spearman rho| and mutual information of each coefficient with z
  figures/pca_by_z.png          first PCs of the normalized coefficients, coloured by z
  figures/sample.png            z and G distributions of the matched XP sample
"""
import matplotlib
matplotlib.use("Agg")
```

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.stats import spearmanr
from sklearn.decomposition import PCA
from sklearn.feature_selection import mutual_info_regression
from sklearn.preprocessing import StandardScaler

from common import DATA, FEATURES, FIG, LINES, N_COEF, SEED, SPECTRA

plt.rcParams.update({"figure.dpi": 130, "font.size": 9})


def sample_plot(df):
    fig, ax = plt.subplots(1, 3, figsize=(11, 3))
    ax[0].hist(df.z, bins=80, color="0.3")
    ax[0].set(xlabel="SDSS z", ylabel="N", title=f"N = {len(df):,}")
    ax[1].hist(df.phot_g_mean_mag, bins=60, color="0.3")
    ax[1].set(xlabel="Gaia G")
    ax[2].scatter(df.z, df.phot_g_mean_mag, s=1, alpha=0.3, c="0.2", rasterized=True)
    ax[2].set(xlabel="z", ylabel="G")
    ax[2].invert_yaxis()
    fig.tight_layout()
    fig.savefig(FIG / "sample.png")


def ladder(spec):
    wl, flux, z = spec["wl"], spec["flux"], spec["z"]
    ok = np.all(np.isfinite(flux), axis=1)
    wl_ok = (wl > 345) & (wl < 1000)   # edges are poorly constrained
    flux, z = flux[ok][:, wl_ok], z[ok]
    wl = wl[wl_ok]
    norm = flux / np.median(flux, axis=1, keepdims=True)

    # equal-count bins in z, so every row of the image has the same S/N
    nb = int(np.clip(len(z) // 40, 30, 150))
    edges = np.quantile(z, np.linspace(0, 1, nb + 1))
    idx = np.clip(np.searchsorted(edges, z, side="right") - 1, 0, nb - 1)
    stack = np.vstack([np.median(norm[idx == i], axis=0) for i in range(nb)])
    zc = np.array([np.median(z[idx == i]) for i in range(nb)])
    rel = stack / np.median(stack, axis=0, keepdims=True)   # divide out the mean observed shape

    fig, ax = plt.subplots(1, 2, figsize=(12, 6), gridspec_kw={"width_ratios": [3, 2]})
    lim = np.nanpercentile(np.abs(np.log(np.clip(rel, 1e-3, None))), 98)
    dw = wl[1] - wl[0]
    ax[0].imshow(np.log(np.clip(rel, 1e-3, None)), cmap="RdBu_r", vmin=-lim, vmax=lim, origin="lower", aspect="auto",
                 extent=[wl[0] - dw / 2, wl[-1] + dw / 2, 0, nb], interpolation="nearest")
    for name, lam in LINES.items():
        track = lam * (1 + zc)
        inside = (track > wl[0]) & (track < wl[-1])
        if inside.sum() > 1:
            ax[0].plot(track[inside], np.arange(nb)[inside] + 0.5, "k--", lw=0.7)
            j = np.flatnonzero(inside)[len(np.flatnonzero(inside)) // 2]
            ax[0].text(track[j] + 4, j + 0.5, name, fontsize=8)
    ticks = np.linspace(0, nb - 1, 8).astype(int)
    ax[0].set_yticks(ticks + 0.5, [f"{zc[t]:.2f}" for t in ticks])
    ax[0].set(xlabel="observed wavelength [nm]", ylabel="SDSS z (equal-count bins)",
              title="Gaia XP quasar spectra, stacked by z, relative to the mean shape (log ratio)")

    for k in np.linspace(0, nb - 1, 6).astype(int):
        ax[1].plot(wl, stack[k] / np.median(stack[k]), label=f"z≈{zc[k]:.2f}")
    ax[1].set(xlabel="observed wavelength [nm]", ylabel="normalized flux", title="example stacks")
    ax[1].legend(fontsize=7)
    fig.tight_layout()
    fig.savefig(FIG / "redshift_ladder.png", dpi=160)


def coef_relevance(df):
    cols = [f"c_{b}_{k:02d}" for b in ("bp", "rp") for k in range(N_COEF)]
    X = df[cols].to_numpy()
    rho = np.array([spearmanr(X[:, j], df.z).statistic for j in range(X.shape[1])])
    sub = df.sample(min(len(df), 20000), random_state=SEED)
    mi = mutual_info_regression(sub[cols].to_numpy(), sub.z, random_state=SEED)
    # same for the broadband colours, as the reference level
    mi_col = mutual_info_regression(sub[["bp_rp", "bp_g", "g_rp"]].to_numpy(), sub.z, random_state=SEED)
    pd.DataFrame({"coef": cols, "spearman": rho, "mutual_info": mi}).to_csv(DATA / "coef_relevance.csv", index=False)

    fig, ax = plt.subplots(2, 1, figsize=(11, 5), sharex=True)
    x = np.arange(len(cols))
    for a, vals, lab in ((ax[0], np.abs(rho), "|Spearman ρ| with z"), (ax[1], mi, "mutual information [nats]")):
        a.bar(x[:N_COEF], vals[:N_COEF], color="#2b6cb0", label="BP")
        a.bar(x[N_COEF:], vals[N_COEF:], color="#c53030", label="RP")
        a.set_ylabel(lab)
    for v, name in zip(mi_col, ["BP–RP", "BP–G", "G–RP"]):
        ax[1].axhline(v, ls=":", lw=0.8, c="k")
        ax[1].text(len(cols) - 1, v, name, fontsize=7, ha="right", va="bottom")
    ax[0].legend()
    ax[1].set_xticks(np.r_[0:N_COEF:5, N_COEF:2 * N_COEF:5], [f"{k}" for k in np.r_[0:N_COEF:5, 0:N_COEF:5]])
    ax[1].set_xlabel("Hermite coefficient index (BP | RP)")
    fig.tight_layout()
    fig.savefig(FIG / "coef_vs_z.png")
    order = np.argsort(mi)[::-1][:10]
    print("Top-10 coefficients by MI:", [(cols[j], round(mi[j], 3)) for j in order])
    print("MI of colours BP-RP, BP-G, G-RP:", mi_col.round(3))
def pca_plot(df):
    cols = [c for c in df if c.startswith("c_")]
```

```
    Z = PCA(n_components=6, random_state=SEED).fit_transform(StandardScaler().fit_transform(df[cols]))
    fig, ax = plt.subplots(1, 3, figsize=(12, 3.8))
    order = np.argsort(df.z.to_numpy())
    for a, (i, j) in zip(ax, [(0, 1), (1, 2), (2, 3)]):
        sc = a.scatter(Z[order, i], Z[order, j], c=df.z.to_numpy()[order], s=1.5, cmap="turbo", rasterized=True)
        a.set(xlabel=f"PC{i + 1}", ylabel=f"PC{j + 1}")
        lo, hi = np.percentile(Z[:, [i, j]], [0.5, 99.5], axis=0)
        a.set(xlim=(lo[0], hi[0]), ylim=(lo[1], hi[1]))
    fig.colorbar(sc, ax=ax, label="SDSS z")
    fig.savefig(FIG / "pca_by_z.png")


def main():
    df = pd.read_parquet(FEATURES)
    spec = np.load(SPECTRA)
    sample_plot(df)
    ladder(spec)
    coef_relevance(df)
    pca_plot(df)
    print("figures written to", FIG)


if __name__ == "__main__":
    main()
```

05_regress.py

```
"""Step 5: how much redshift information is in the XP coefficients, beyond broadband colours?

Feature sets (all include G magnitude, which carries some z information via the luminosity function):
  A_colours   : BP-RP, BP-G, G-RP, G
  B_xp        : 110 flux-normalized XP coefficients, G
  B_xp_low    : only the first 20 BP + 20 RP coefficients (robustness to noisy high orders), G
Models: kNN, LightGBM, MLP.  5-fold CV, fixed seed, target log(1+z).
Benchmark C: Gaia DR3 QSOC redshift (evaluated on the subset where it exists).
Control: LightGBM on B_xp with shuffled z (must have no skill).
"""
import matplotlib

matplotlib.use("Agg")
import lightgbm as lgb
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.model_selection import KFold
from sklearn.neighbors import KNeighborsRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import QuantileTransformer, StandardScaler

from common import DATA, FEATURES, FIG, N_COEF, PRED, SEED, confusion_ratios, metrics

plt.rcParams.update({"figure.dpi": 130, "font.size": 9})


def feature_sets(df):
    xp = [f"c_{b}_{k:02d}" for b in ("bp", "rp") for k in range(N_COEF)]
    xp_low = [f"c_{b}_{k:02d}" for b in ("bp", "rp") for k in range(20)]
    return {
        "A_colours": ["bp_rp", "bp_g", "g_rp", "phot_g_mean_mag"],
        "B_xp": xp + ["phot_g_mean_mag"],
        "B_xp_low": xp_low + ["phot_g_mean_mag"],
    }


def models(n_feat):
    return {
        "kNN": lambda: make_pipeline(StandardScaler(), *( [PCA(n_components=30, whiten=True, random_state=SEED)]
                                                          if n_feat > 30 else []),
                                     KNeighborsRegressor(n_neighbors=15, weights="distance")),
        "LightGBM": lambda: lgb.LGBMRegressor(n_estimators=1500, learning_rate=0.03, num_leaves=63,
                                              min_child_samples=20, subsample=0.8, subsample_freq=1,
                                              colsample_bytree=0.7, random_state=SEED, verbose=-1),
        # quantile -> Gaussian scaling: flux-normalised coefficients are heavy-tailed, which broke
        # StandardScaler + MLP on the 3k subset (sigma_NMAD 0.21, no better than colours)
        "MLP": lambda: make_pipeline(QuantileTransformer(output_distribution="normal", random_state=SEED),
                                     MLPRegressor(hidden_layer_sizes=(256, 128, 64), alpha=1e-3,
                                                  early_stopping=True, max_iter=600, random_state=SEED)),
    }


def cv_predict(make, X, y, folds):
    pred = np.empty_like(y)
    for tr, te in folds:
        mdl = make()
        mdl.fit(X[tr], y[tr])
        pred[te] = mdl.predict(X[te])
    return pred


def main():
    df = pd.read_parquet(FEATURES)
    z = df.z.to_numpy()
    y = np.log1p(z)
```

```
    folds = list(KFold(5, shuffle=True, random_state=SEED).split(df))
    rows, preds = [], {"source_id": df.source_id, "z": z}

    for fs_name, cols in feature_sets(df).items():
        X = df[cols].to_numpy(float)
        for m_name, make in models(len(cols)).items():
            zp = np.expm1(cv_predict(make, X, y, folds))
            key = f"{fs_name}|{m_name}"
            preds[key] = zp
            rows.append({"features": fs_name, "model": m_name, **metrics(z, zp)})
            print(rows[-1], flush=True)

    # shuffled-z control
    rng = np.random.default_rng(SEED)
    y_shuf = rng.permutation(y)
    X = df[feature_sets(df)["B_xp"]].to_numpy(float)
    zp = np.expm1(cv_predict(models(X.shape[1])["LightGBM"], X, y_shuf, folds))
    rows.append({"features": "B_xp (shuffled z control)", "model": "LightGBM", **metrics(np.expm1(y_shuf), zp)})
    print(rows[-1])

    # Gaia QSOC benchmark on its own subset, with our models on the same subset for a fair comparison
    has = df.redshift_qsoc.notna().to_numpy()
    if has.sum() > 50:
        rows.append({"features": "C_gaia_QSOC", "model": "Gaia DR3 QSOC", **metrics(z[has], df.redshift_qsoc[has])})
        for key in ("A_colours|LightGBM", "B_xp|LightGBM"):
            rows.append({"features": key.split("|")[0] + " (QSOC subset)", "model": "LightGBM",
                         **metrics(z[has], preds[key][has])})
        preds["C_gaia_QSOC"] = df.redshift_qsoc.to_numpy()

    res = pd.DataFrame(rows)
    res.to_csv(DATA / "metrics.csv", index=False)
    pd.DataFrame(preds).to_parquet(PRED)
    with pd.option_context("display.width", 200, "display.max_columns", 20):
        print("\n", res.round(4).to_string(index=False))

    plot_scatter(df, preds, res)
    plot_binned(df, preds)


def plot_scatter(df, preds, res):
    z = df.z.to_numpy()
    best_b = res[res.features == "B_xp"].sort_values("sigma_NMAD").iloc[0].model
    best_a = res[res.features == "A_colours"].sort_values("sigma_NMAD").iloc[0].model
    panels = [(f"A_colours|{best_a}", f"colours only ({best_a})"), (f"B_xp|{best_b}", f"XP coefficients ({best_b})")]
    if "C_gaia_QSOC" in preds:
        panels.append(("C_gaia_QSOC", "Gaia DR3 QSOC"))
    zmax = np.percentile(z, 99.8)
    fig, ax = plt.subplots(1, len(panels), figsize=(4.3 * len(panels), 4.2), squeeze=False)
    zz = np.linspace(0, zmax, 100)
    for a, (key, title) in zip(ax[0], panels):
        zp = np.asarray(preds[key], float)
        ok = np.isfinite(zp)
        a.hexbin(z[ok], zp[ok], gridsize=90, extent=(0, zmax, 0, zmax), bins="log", cmap="Greys", mincnt=1)
        for name, r in confusion_ratios():
            line = (1 + zz) * r - 1
            if np.any((line > 0) & (line < zmax)) and 0.55 < r < 1.8:
                a.plot(zz, line, lw=0.5, alpha=0.6, c="#c53030")
        a.plot(zz, zz, c="#2b6cb0", lw=0.8)
        m = res[(res.features + "|" + res.model) == key]
        m = m.iloc[0] if len(m) else pd.Series(metrics(z[ok], zp[ok]))
        a.set(xlim=(0, zmax), ylim=(0, zmax), xlabel="SDSS z", ylabel="predicted z",
              title=f"{title}\nσ_NMAD={m.sigma_NMAD:.3f}  outl(0.2)={m['outlier_0.2']:.1%}")
    fig.text(0.01, 0.01, "red: loci of line mis-identification (1+z_pred)/(1+z_true)=λ_a/λ_b", fontsize=7)
    fig.tight_layout()
    fig.savefig(FIG / "z_pred_vs_true.png", dpi=160)


def plot_binned(df, preds):
    z = df.z.to_numpy()
    keys = [k for k in preds if "|" in k and (k.endswith("LightGBM"))] + (["C_gaia_QSOC"] if "C_gaia_QSOC" in preds else [])
    fig, ax = plt.subplots(2, 2, figsize=(10, 6), sharex="col")
    for col, (var, label) in enumerate([(z, "SDSS z"), (df.phot_g_mean_mag.to_numpy(), "Gaia G")]):
        edges = np.quantile(var, np.linspace(0, 1, 13))
        mid = 0.5 * (edges[1:] + edges[:-1])
        for k in keys:
            zp = np.asarray(preds[k], float)
            s, o = [], []
            for lo, hi in zip(edges[:-1], edges[1:]):
                sel = (var >= lo) & (var <= hi) & np.isfinite(zp)
                mm = metrics(z[sel], zp[sel]) if sel.sum() > 10 else {"sigma_NMAD": np.nan, "outlier_0.2": np.nan}
                s.append(mm["sigma_NMAD"]); o.append(mm["outlier_0.2"])
            ax[0, col].plot(mid, s, "o-", ms=3, label=k)
            ax[1, col].plot(mid, o, "o-", ms=3, label=k)
        ax[0, col].set(ylabel="σ_NMAD", yscale="log")
        ax[1, col].set(ylabel="outlier fraction (|Δz|/(1+z)>0.2)", xlabel=label)
    ax[0, 0].legend(fontsize=7)
    fig.tight_layout()
    fig.savefig(FIG / "metrics_binned.png")


if __name__ == "__main__":
    main()
```

06_pz.py

```
"""Step 6: probabilistic redshifts p(z) from the XP coefficients, to expose line-confusion degeneracies.

MLP classifier over bins in log(1+z), 5-fold CV -> p(z) for every quasar.
  figures/pz_examples.png     p(z) for well-measured, bimodal and catastrophic cases
  figures/pz_calibration.png  PIT histogram + coverage, peak multiplicity
  data/pz_summary.parquet     mode, median, 68% interval, n_peaks, PIT per quasar
"""
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.signal import find_peaks
from sklearn.model_selection import StratifiedKFold
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import QuantileTransformer

from common import DATA, FEATURES, FIG, N_COEF, SEED, metrics

plt.rcParams.update({"figure.dpi": 130, "font.size": 9})
NBINS = 60


def main():
    df = pd.read_parquet(FEATURES)
    cols = [f"c_{b}_{k:02d}" for b in ("bp", "rp") for k in range(N_COEF)] + ["phot_g_mean_mag"]
    X = df[cols].to_numpy(float)
    z = df.z.to_numpy()
    u = np.log1p(z)
    edges = np.linspace(u.min() - 1e-6, u.max() + 1e-6, NBINS + 1)
    lab = np.clip(np.searchsorted(edges, u, side="right") - 1, 0, NBINS - 1)
    # merge empty classes away: LightGBM needs every label to exist in training data
    present = np.unique(lab)
    remap = {c: i for i, c in enumerate(present)}
    y = np.array([remap[c] for c in lab])
    centers = np.expm1(0.5 * (edges[present] + edges[present + 1]))
    widths = np.expm1(edges[present + 1]) - np.expm1(edges[present])
    upper = np.expm1(edges[present + 1])   # the CDF evaluated at bin upper edges

    P = np.zeros((len(df), len(present)))
    strat = np.minimum(y, np.percentile(y, 98).astype(int))   # avoid tiny strata
    for k, (tr, te) in enumerate(StratifiedKFold(5, shuffle=True, random_state=SEED).split(X, strat)):
        # LightGBM multiclass diverged here (one-hot probabilities, 33% outliers); an MLP classifier
        # on quantile-scaled features (same scaling as the best regressor) trains stably with log-loss
        mdl = make_pipeline(QuantileTransformer(output_distribution="normal", random_state=SEED),
                            MLPClassifier(hidden_layer_sizes=(256, 128), alpha=1e-3, early_stopping=True,
                                          max_iter=400, random_state=SEED))
        mdl.fit(X[tr], y[tr])
        P[np.ix_(te, mdl[-1].classes_)] = mdl.predict_proba(X[te])
        print(f"fold {k + 1}/5 done", flush=True)

    pdf = P / widths            # density per unit z
    cdf = np.cumsum(P, axis=1)
    z_mode = centers[np.argmax(P, axis=1)]
    z_med = np.array([np.interp(0.5, c, upper) for c in cdf])
    lo = np.array([np.interp(0.16, c, upper) for c in cdf])
    hi = np.array([np.interp(0.84, c, upper) for c in cdf])
    pit = np.array([np.interp(zt, upper, c) for zt, c in zip(z, cdf)])
    n_peaks = np.array([len(find_peaks(np.r_[0, p, 0], height=0.1 * p.max(), prominence=0.05 * p.max())[0])
                        for p in P])
    summ = pd.DataFrame({"source_id": df.source_id, "z": z, "z_mode": z_mode, "z_median": z_med,
                         "z_lo68": lo, "z_hi68": hi, "pit": pit, "n_peaks": n_peaks})
    summ.to_parquet(DATA / "pz_summary.parquet")
    np.savez_compressed(DATA / "pz_grid.npz", centers=centers, P=P.astype(np.float32))

    print("mode   :", metrics(z, z_mode))
    print("median :", metrics(z, z_med))
    print("fraction with >1 significant peak:", np.mean(n_peaks > 1).round(3))
    cov = np.mean((z >= lo) & (z <= hi))
    print(f"68% interval coverage: {cov:.3f}")
    outl = np.abs(z_mode - z) / (1 + z) > 0.2
    if outl.any():
        # for catastrophic outliers, how much probability did the model put near the truth?
        near = np.array([P[i, np.abs(centers - z[i]) / (1 + z[i]) < 0.05].sum() for i in np.flatnonzero(outl)])
        print(f"catastrophic outliers (mode): {outl.mean():.3f}; median p(|z-z_true|<0.05(1+z)) for them: "
              f"{np.median(near):.2f}; >0.2 for {np.mean(near > 0.2):.1%}")

    # ---- figures ----
    fig, ax = plt.subplots(1, 3, figsize=(12, 3.5))
    ax[0].hist(pit, bins=20, color="0.3")
    ax[0].axhline(len(pit) / 20, c="r", lw=0.8)
    ax[0].set(xlabel="PIT", title="PIT (flat = calibrated)")
    ax[1].hist(n_peaks, bins=np.arange(0.5, 6.5), color="0.3")
    ax[1].set(xlabel="number of significant p(z) peaks")
    ax[2].hexbin(z, z_mode, gridsize=80, bins="log", cmap="Greys", mincnt=1)
    ax[2].plot([0, z.max()], [0, z.max()], c="#2b6cb0", lw=0.8)
    ax[2].set(xlabel="SDSS z", ylabel="p(z) mode")
    fig.tight_layout()
    fig.savefig(FIG / "pz_calibration.png")

    rng = np.random.default_rng(SEED)
    groups = {
        "single peak, correct": np.flatnonzero((n_peaks == 1) & ~outl),
        "multi-peak": np.flatnonzero(n_peaks > 1),
```

```
        "catastrophic (mode)": np.flatnonzero(outl),
    }
    fig, ax = plt.subplots(3, 4, figsize=(12, 7), sharex=True)
    for r, (name, idx) in enumerate(groups.items()):
        pick = rng.choice(idx, size=min(4, len(idx)), replace=False) if len(idx) else []
        for c, i in enumerate(pick):
            a = ax[r, c]
            a.plot(centers, pdf[i], c="k", lw=1)
            a.axvline(z[i], c="#c53030", lw=1)
            a.set_title(f"{name}  G={df.phot_g_mean_mag.iloc[i]:.1f}", fontsize=8)
        ax[r, 0].set_ylabel("p(z)")
    for a in ax[-1]:
        a.set_xlabel("z  (red: SDSS)")
    fig.tight_layout()
    fig.savefig(FIG / "pz_examples.png")


if __name__ == "__main__":
    main()
```

make_report.py

```
"""Build REPORT.pdf: what was done, results, prior work, caveats, and full source listings."""
from datetime import date
from pathlib import Path

import pandas as pd
from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.lib.units import cm
from reportlab.lib.utils import ImageReader
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.platypus import (Image, KeepTogether, PageBreak, Paragraph, Preformatted, SimpleDocTemplate,
                                Spacer, Table, TableStyle)

ROOT = Path(__file__).resolve().parent
FIG, DATA = ROOT / "figures", ROOT / "data"
FONT_DIR = Path("/usr/share/fonts/truetype/dejavu")
for name, f in [("DV", "DejaVuSans.ttf"), ("DV-B", "DejaVuSans-Bold.ttf"), ("DV-I", "DejaVuSerif.ttf"),  # no DejaVu Sans Oblique installed: s
                ("DVMono", "DejaVuSansMono.ttf")]:
    pdfmetrics.registerFont(TTFont(name, str(FONT_DIR / f)))
pdfmetrics.registerFontFamily("DV", normal="DV", bold="DV-B", italic="DV-I", boldItalic="DV-B")

ss = getSampleStyleSheet()
body = ParagraphStyle("body", parent=ss["Normal"], fontName="DV", fontSize=9.5, leading=13.2, spaceAfter=5)
bullet = ParagraphStyle("bullet", parent=body, leftIndent=12, bulletIndent=2, spaceAfter=2)
h1 = ParagraphStyle("h1", parent=ss["Heading1"], fontName="DV-B", fontSize=14, spaceBefore=10, spaceAfter=6)
h2 = ParagraphStyle("h2", parent=ss["Heading2"], fontName="DV-B", fontSize=11, spaceBefore=8, spaceAfter=4)
title = ParagraphStyle("title", parent=ss["Title"], fontName="DV-B", fontSize=18, leading=22)
sub = ParagraphStyle("sub", parent=body, alignment=TA_CENTER, textColor=colors.HexColor("#555555"))
cap = ParagraphStyle("cap", parent=body, fontSize=8.3, leading=11, textColor=colors.HexColor("#333333"), spaceAfter=10)
code = ParagraphStyle("code", fontName="DVMono", fontSize=6.3, leading=7.6)
cell = ParagraphStyle("cell", parent=body, fontSize=7.4, leading=9, spaceAfter=0)

W = A4[0] - 3.6 * cm
story = []


def P(text, style=body):
    story.append(Paragraph(text, style))


def bullets(items):
    for it in items:
        story.append(Paragraph(it, bullet, bulletText="•"))


def fig(name, caption, width=W):
    path = FIG / name
    iw, ih = ImageReader(str(path)).getSize()
    img = Image(str(path), width=width, height=width * ih / iw)
    story.append(KeepTogether([img, Paragraph(caption, cap)]))


# ---------------------------------------------------------------- title + summary
story.append(Paragraph("Quasar redshifts from Gaia DR3 XP coefficients", title))
P(f"Project report · {date.today():%d %B %Y} · prepared with Claude Code", sub)
story.append(Spacer(1, 10))

story.append(Paragraph("Summary", h1))
P("<b>Question.</b> Do the coefficients of Gaia DR3 BP/RP (XP) spectra carry information on quasar "
  "redshift? SDSS DR16Q spectroscopic redshifts were matched to Gaia DR3 sources with published XP "
  "coefficients, and redshift was predicted from the coefficients with cross-validated models.")
bullets([
    "<b>Yes, strongly.</b> On 36,402 quasars, a kNN on the first 20 BP + 20 RP coefficients reaches "
    "σ<sub>NMAD</sub> = 0.0040 in Δz/(1+z) with 88% of objects within 0.01(1+z); an MLP reaches 0.50% "
    "catastrophic outliers (|Δz|/(1+z) &gt; 0.2). Gaia colours alone give σ<sub>NMAD</sub> ≈ 0.18 and ≈ 29% outliers.",
    "The signal is visible directly: stacked spectra show Lyα, CIV, CIII], MgII, Hβ+[OIII] and Hα moving "
    "along λ<sub>rest</sub>(1+z).",
    "Gaia's own QSOC redshifts (template χ² fit) remain the most precise (σ<sub>NMAD</sub> = 0.0030) but have more "
    "catastrophic outliers (1.4%) than the best MLP (0.50%).",
    "<b>Not novel.</b> Refereed work already estimates quasar redshifts from XP spectra with SDSS-trained "
```

```
    "models, including directly from the Hermite coefficients (Sect. 6). The study was therefore not "
    "developed into a paper.",
])

# ---------------------------------------------------------------- approach
story.append(Paragraph("1  Approach", h1))
P("Quasar broad emission lines cross the 330–1050 nm XP window as redshift changes. Even at XP's very low "
  "resolution, their positions and the continuum shape should be encoded in the 55+55 Hermite "
  "coefficients. The test has two parts: (i) <i>see</i> the signal in the data, and (ii) <i>measure</i> how much "
  "redshift information the coefficients carry beyond broadband Gaia colours, against Gaia's QSOC as "
  "an external benchmark and a shuffled-redshift control.")

# ---------------------------------------------------------------- data
story.append(Paragraph("2  Data", h1))
story.append(Paragraph("2.1  Cross-match (01_crossmatch.py)", h2))
P("SDSS DR16Q (750,414 quasars) lists a Gaia DR2 counterpart for 469,774 of them. DR2 ids of the "
  "301,346 counterparts with G<sub>DR2</sub> &lt; 20.5 were uploaded to the Gaia archive (ESA TAP), mapped to "
  "DR3 with <font face='DVMono'>gaiadr3.dr2_neighbourhood</font>, joined to <font face='DVMono'>gaia_source</font>, "
  "and the DR3 neighbour closest to the SDSS position was kept (&lt; 1″; median separation 0.048″). "
  "QSOC redshifts come from <font face='DVMono'>gaiadr3.qso_candidates</font>.")
P("A direct 1″ positional join was tried first, but the archive aborts anonymous jobs after 60 s, which "
  "made it impractically slow. It did complete for the first 190,000 DR16Q rows, which serves as a "
  "completeness check: the id-based route recovers 6,392 of the 6,395 XP sources found positionally "
  "(99.95%), always the same DR3 source.")
t = [["DR16Q quasars", "750,414"], ["with Gaia DR3 counterpart < 1″ (G_DR2 < 20.5)", "301,344 (40.2%)"],
     ["… with XP coefficients", "36,404 (4.85%)"], ["… with Gaia QSOC redshift", "36,356"],
     ["usable after cleaning", "36,402"], ["G (5 / 50 / 95 percentile)", "17.32 / 18.56 / 18.97"],
     ["z (5 / 50 / 95 percentile)", "0.43 / 1.50 / 2.67"]]
tb = Table(t, colWidths=[W * 0.62, W * 0.3])
tb.setStyle(TableStyle([("FONT", (0, 0), (-1, -1), "DV", 8.5), ("LINEBELOW", (0, 0), (-1, -1), 0.25, colors.lightgrey),
                        ("ALIGN", (1, 0), (1, -1), "RIGHT")]))
story.append(tb)
story.append(Spacer(1, 6))
P("Unlike the generic G &lt; 17.65 limit for published XP spectra, 92% of the matched quasars with XP are "
  "fainter than that, down to G ≈ 19–20.")
fig("sample.png", "<b>Fig. 1.</b> Matched sample: SDSS redshift and Gaia G distributions.")

story.append(Paragraph("2.2  XP coefficients (02_fetch_xp.py)", h2))
P("ESA's TAP service does not expose the XP coefficient table, and its DataLink service limits anonymous "
  "users to 50 sources per request at ~5 s of server time per source (~1,000 spectra per 25 min). The "
  "coefficients were instead retrieved from the AIP Gaia mirror (gaia.aip.de), whose TAP service has "
  "<font face='DVMono'>gaiadr3.xp_continuous_mean_spectrum</font>: all 36,404 spectra in ~8 minutes. "
  "For 4,000 spectra fetched from both services, the coefficients are bit-identical and the errors and "
  "correlations agree to float32 rounding (≤ 6×10<super>−8</super> relative).")

# ---------------------------------------------------------------- methods
story.append(Paragraph("3  Methods", h1))
bullets([
    "<b>Features (03_features.py).</b> The 110 coefficients divided by the G-band flux. They are linear in flux, so "
    "this leaves a pure shape vector. Calibrated, sampled spectra (336–1020 nm, GaiaXPy) are used only for "
    "visualisation.",
    "<b>Feature sets.</b> A: BP−RP, BP−G, G−RP, G. B: all 110 normalised coefficients + G. "
    "B<sub>low</sub>: first 20 BP + 20 RP coefficients + G. G is included everywhere, since magnitude "
    "carries some redshift information through the luminosity function.",
    "<b>Models (05_regress.py).</b> kNN (k = 15, distance-weighted, standardised + 30-component whitened PCA), "
    "LightGBM (1,500 trees), MLP (256-128-64 units, quantile-to-Gaussian input scaling). Target log(1+z). "
    "5-fold CV with a fixed seed; every prediction is out-of-fold.",
    "<b>Metrics.</b> Δz/(1+z); σ<sub>NMAD</sub> = 1.4826 × median absolute deviation; outlier fractions at "
    "0.1 and 0.2; fraction within 0.01(1+z).",
    "<b>Controls.</b> LightGBM on shuffled redshifts; Gaia QSOC on the same objects.",
    "<b>p(z) (06_pz.py).</b> MLP classifier over 56 bins in log(1+z), 5-fold CV. Gives the mode, median, 68% "
    "interval, number of significant peaks and the probability integral transform (PIT).",
])

# ---------------------------------------------------------------- results
story.append(Paragraph("4  Results", h1))
story.append(Paragraph("4.1  The signal is visible", h2))
fig("redshift_ladder.png", "<b>Fig. 2.</b> Left: calibrated XP spectra stacked in equal-count redshift bins, "
    "divided by the mean observed shape (log ratio). Dashed: λ<sub>rest</sub>(1+z) tracks. Lyα, CIV, CIII], MgII, "
    "Hβ+[OIII] and Hα follow their tracks; Lyα-forest absorption shows blueward of Lyα at high z. Faint "
    "parallel stripes are partly weaker lines and partly basis-function ringing. The vertical seam near "
    "640 nm at low z is the BP/RP join. Right: example stacks.")
fig("coef_vs_z.png", "<b>Fig. 3.</b> |Spearman ρ| and mutual information (MI) of each coefficient with z. "
    "Dotted lines: MI of the colours (0.23, 0.37, 0.24 nats). Several single coefficients carry 0.8–1.0 nats, "
    "mostly low-order RP terms.")
fig("pca_by_z.png", "<b>Fig. 4.</b> First principal components of the normalised coefficients, coloured by "
    "redshift.", width=W * 0.95)

story.append(Paragraph("4.2  How much information: cross-validated regression", h2))
m = pd.read_csv(DATA / "metrics.csv")
hdr = ["Features", "Model", "N", "bias", "σ<sub>NMAD</sub>", "outl. 0.1", "outl. 0.2", "within 0.01(1+z)"]
rows = [hdr] + [[r.features, r.model, f"{r.N:,}", f"{r.bias_median:+.4f}", f"{r.sigma_NMAD:.4f}",
                 f"{r['outlier_0.1']:.1%}", f"{r['outlier_0.2']:.1%}", f"{r['frac_|dz|<0.01']:.1%}"]
                for _, r in m.iterrows()]
rows = [[Paragraph(str(c), cell) for c in row] for row in rows]
tb = Table(rows, colWidths=[W * x for x in (0.25, 0.14, 0.09, 0.1, 0.09, 0.09, 0.09, 0.15)], repeatRows=1)
tb.setStyle(TableStyle([("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#e8edf3")),
                        ("LINEBELOW", (0, 0), (-1, -1), 0.25, colors.lightgrey),
                        ("VALIGN", (0, 0), (-1, -1), "MIDDLE")]))
story.append(tb)
P("<b>Table 1.</b> 5-fold cross-validated metrics (data/metrics.csv). The last two rows rerun the models on the "
  "36,355 objects that also have a QSOC redshift.", cap)
bullets([
    "Coefficients beat colours by a factor 20–40 in σ<sub>NMAD</sub> and up to ~60 in catastrophic outliers. "
    "Colours cannot separate redshifts between z ≈ 1 and 2.5: predictions there pile up near 1.7.",
    "kNN on the low-order coefficients is the sharpest (σ<sub>NMAD</sub> 0.0040), close to QSOC. Dropping the "
```

```
    "noisy high-order coefficients matters for kNN, much less for trees.",
    "The MLPs give the fewest catastrophic outliers (0.50–0.59%), about 3× fewer than QSOC. Plain "
    "standardisation broke the MLP (on 3,000 objects it was no better than colours); quantile scaling fixed it.",
    "The shuffled-redshift control has no skill (σ<sub>NMAD</sub> 0.29, 52% outliers).",
    "With 3,000 objects (an early random subset), the scores were much worse (kNN on all coefficients "
    "σ<sub>NMAD</sub> 0.074 vs 0.011 with 36k): these models are data-limited.",
])
fig("z_pred_vs_true.png", "<b>Fig. 5.</b> Predicted vs SDSS redshift: colours only, XP coefficients (MLP), Gaia QSOC. "
    "Red lines: loci where one emission line is mistaken for another, (1+z<sub>pred</sub>)/(1+z<sub>true</sub>) = "
    "λ<sub>a</sub>/λ<sub>b</sub>. QSOC's remaining outliers include a cluster of z ≈ 0.5–1 quasars placed near z ≈ 3.5.")
fig("metrics_binned.png", "<b>Fig. 6.</b> σ<sub>NMAD</sub> and outlier fraction vs SDSS redshift and Gaia G "
    "(LightGBM models and QSOC). All methods degrade smoothly towards G ≈ 19.5.")

story.append(Paragraph("4.3  Redshift probabilities and line confusion", h2))
P("The p(z) classifier gives 0.63% outliers (mode) and σ<sub>NMAD</sub> 0.011 (median). 2.3% of objects get a "
  "p(z) with more than one significant peak. For 43% of the remaining catastrophic outliers, the model "
  "still puts more than 20% probability within 0.05(1+z) of the true redshift, so most of its failures are "
  "flagged ambiguities. The 68% intervals are too wide (81% coverage), probably because the "
  "bins are coarse. A first attempt with a LightGBM multiclass model diverged (one-hot probabilities, "
  "33% outliers) and was replaced.")
fig("pz_examples.png", "<b>Fig. 7.</b> Example p(z): single-peaked correct, multi-peaked, and catastrophic "
    "cases. Red: SDSS redshift.")
fig("pz_calibration.png", "<b>Fig. 8.</b> PIT histogram, number of significant peaks, and p(z) mode vs SDSS z.")

# ---------------------------------------------------------------------- prior work
story.append(Paragraph("5  Caveats", h1))
bullets([
    "The sample is DR16Q-selected and relatively bright (G ≤ 19.5). Its performance does not carry over to "
    "fainter or differently selected quasars, or to contamination by stars and galaxies, which was not "
    "modelled here.",
    "Cross-validation is within DR16Q only. No independent test sample (e.g. DESI) was used, and no "
    "spatial-block split was tried.",
    "QSOC's template was built from SDSS DR12Q quasars, so the QSOC comparison is not fully independent "
    "of SDSS. The comparison with published deep-learning redshifts was not done (Sect. 6).",
    "Hyperparameters were not tuned. The p(z) intervals are not calibrated.",
])

story.append(Paragraph("6  Prior work and assessment", h1))
P("A literature search found refereed work that already estimates quasar redshifts from Gaia XP spectra "
  "with models trained on SDSS. Only abstracts and summaries were read, because the full texts were not "
  "reachable from the analysis environment.")
bullets([
    "Delchambre et al. 2023, A&amp;amp;A 674, A31 — Gaia DR3 QSOC: χ² fit of a composite quasar template (from SDSS "
    "DR12Q) to BP/RP spectra.",
    "Storey-Fisher et al. 2024, ApJ — Quaia: kNN on QSOC redshifts plus Gaia/unWISE photometry, trained on SDSS; "
    "reports outliers from line aliasing.",
    "Fu et al. 2024, ApJS 271, 54 (CatNorth) and Fu et al. 2025, ApJS 279, 54 (CatSouth) — a CNN on sampled XP "
    "spectra, trained on SDSS (CatSouth reports σ<sub>NMAD</sub> ≈ 0.030, outliers ≈ 2.5%).",
    "“Unveiling Quasars in Gaia DR3: Multimodal Classification and Redshift Estimation of BP/RP Sources with "
    "Deep Learning”, ApJS, doi:10.3847/1538-4365/ae4003 — a deep network on the Hermite coefficients, sampled "
    "fluxes and catalogue features (SpecNet-Z); catalogue on Zenodo (10.5281/zenodo.17960118).",
])
P("The core idea is therefore established. The only potentially new point, that a simple kNN on "
  "low-order coefficients may be much sharper than the published CNNs (0.004 vs ≈ 0.03), would need a "
  "comparison on identical objects. That comparison must account for the published models having been "
  "trained on SDSS quasars that overlap this sample. It was not pursued.")

# ---------------------------------------------------------------------- reproducibility
story.append(Paragraph("7  Reproducibility", h1))
P("All code is in <font face='DVMono'>gaia_qso_z/</font> and listed in full in the Appendix: "
  "<font face='DVMono'>common.py</font>, <font face='DVMono'>01_crossmatch.py</font> … "
  "<font face='DVMono'>06_pz.py</font>, <font face='DVMono'>run_all.sh</font>, "
  "<font face='DVMono'>make_report.py</font> (this report), <font face='DVMono'>requirements.txt</font> and "
  "<font face='DVMono'>README.md</font>. To reproduce: <font face='DVMono'>pip install -r requirements.txt</font> "
  "then <font face='DVMono'>./run_all.sh</font>. Network access is needed to data.sdss.org (DR16Q, 1.7 GB), "
  "gea.esac.esa.int (Gaia TAP) and gaia.aip.de (XP coefficients). The full run takes about an hour on 4 CPUs, "
  "dominated by the cross-match (~10 min) and model training. Raw data products (~3 GB) are not part of the code "
  "and are regenerated by the scripts.")
P("Python 3.13 with: " + ", ".join((ROOT / "requirements.txt").read_text().split()) + ".")

# ---------------------------------------------------------------------- appendix: code
story.append(PageBreak())
story.append(Paragraph("Appendix: source code", h1))
for name in ["README.md", "requirements.txt", "run_all.sh", "common.py", "01_crossmatch.py", "02_fetch_xp.py",
             "03_features.py", "04_explore.py", "05_regress.py", "06_pz.py", "make_report.py"]:
    story.append(Paragraph(name, h2))
    story.append(Preformatted((ROOT / name).read_text(), code))


def footer(canvas, doc):
    canvas.saveState()
    canvas.setFont("DV", 7.5)
    canvas.setFillColor(colors.grey)
    canvas.drawString(1.8 * cm, 1.1 * cm, "Quasar redshifts from Gaia DR3 XP coefficients — project report")
    canvas.drawRightString(A4[0] - 1.8 * cm, 1.1 * cm, f"{doc.page}")
    canvas.restoreState()


doc = SimpleDocTemplate(str(ROOT / "REPORT.pdf"), pagesize=A4, leftMargin=1.8 * cm, rightMargin=1.8 * cm,
                        topMargin=1.6 * cm, bottomMargin=1.8 * cm, title="Quasar redshifts from Gaia DR3 XP coefficients",
                        author="prepared with Claude Code")
doc.build(story, onFirstPage=footer, onLaterPages=footer)
print("wrote", ROOT / "REPORT.pdf")
```